



Mecanismo de Detecção de Golpes e Fraudes Digitais
em Redes Sociais e Mensagerias

DOCUMENTAÇÃO TÉCNICA

Versão 1.0

dAurora | Anatel

Instituto de Informática — Universidade Federal de Goiás

Goiânia — Goiás — Brasil

2026

Sumário

1. Visão Geral da Arquitetura

2. Tecnologias Utilizadas

- 2.1 Frontend
- 2.2 Backend
- 2.3 Módulo de IA
- 2.4 Scraping
- 2.5 Infraestrutura

3. Estrutura do Repositório

- 3.1 Frontend
- 3.2 Backend
- 3.3 Módulo de IA

4. Configuração do Ambiente

- 4.1 Pré-requisitos
- 4.2 Variáveis de Ambiente
- 4.3 Executando o Projeto

5. Banco de Dados

- 5.1 Modelo de Dados
- 5.2 Tabelas

6. API Backend

- 6.1 Casos
- 6.2 Análises
- 6.3 Análise Manual
- 6.4 Parâmetros de Filtro

7. Módulo de Inteligência Artificial

- 7.1 Pipeline de Análise
- 7.2 Classificação de Risco
- 7.3 Tipos de Golpe Suportados
- 7.4 Métricas de Avaliação
- 7.5 Endpoints da API de IA
- 7.6 Exemplo de Resposta
- 7.7 Parametrização (config.json)
- 7.8 Expandindo a Base de Conhecimento

8. Módulo de Scraping

- 8.1 Funcionamento
- 8.2 Autenticação
- 8.3 Configuração dos Grupos Monitorados

9. Segurança e Boas Práticas

- 9.1 Variáveis de Ambiente
- 9.2 CORS
- 9.3 Healthchecks

10. Estratégia de Branches

11. Glossário Técnico

1. Visão Geral da Arquitetura

O Discovery é uma plataforma web para detecção de golpes e fraudes digitais, desenvolvida em arquitetura de microsserviços com cinco componentes orquestrados via Docker Compose.

Fluxo principal:

- O módulo de Scraping coleta mensagens de canais de mensageria(atualmente telegram) automaticamente a cada hora.
- As mensagens são enviadas ao Backend via API REST.
- O Backend repassa o conteúdo ao módulo de IA para análise.
- A IA classifica o conteúdo usando ML local, análise de links, RAG e o modelo Sabiá (Maritaca AI).
- O resultado é armazenado no banco de dados PostgreSQL.
- O Frontend Next.js consome a API do Backend e exibe os dados em tempo real.

Serviço	Tecnologia	Porta	Responsabilidade
frontend	Next.js 16	3000	Interface web para os usuários
backend	FastAPI (Python 3.10)	8000	API REST, regras de negócio, persistência
ia	FastAPI (Python 3.11)	8001	Pipeline de análise com ML e LLM
scraping	Python 3.10 + Telethon	—	Coleta automática de dados do Telegram
db	PostgreSQL 15	5432	Persistência de dados

2. Tecnologias Utilizadas

2.1 Frontend

Tecnologia	Versão	Finalidade
Next.js	16.1.6	Framework React com App Router
React	18	Biblioteca de interface de usuário
Recharts	—	Gráficos interativos (velocímetro, rosca, barras)
jsPDF	—	Geração de relatórios PDF no navegador
Font Awesome	6.5.0	Ícones da interface
CSS Modules / CSS puro	—	Estilização dos componentes

2.2 Backend

Tecnologia	Versão	Finalidade
FastAPI	0.111.0	Framework web assíncrono

SQLAlchemy	2.0.30	ORM para acesso ao banco de dados
Pydantic	2.7.1	Validação de dados e schemas
Alembic	1.13.1	Migrações do banco de dados
psycopg2-binary	2.9.9	Driver de conexão PostgreSQL
httpx	0.27.0	Cliente HTTP assíncrono para comunicação com IA
python-dotenv	1.0.1	Gerenciamento de variáveis de ambiente
uvicorn	0.29.0	Servidor ASGI para FastAPI

2.3 Módulo de IA

Tecnologia	Finalidade	Custo
scikit-learn	Classificador ML (Logistic Regression + TF-IDF)	Gratuito (local)
NLTK + RSLPStemmer	Pré-processamento de texto em português	Gratuito (local)
sentence-transformers	Embeddings multilíngues para o RAG	Gratuito (local)
FAISS (Meta AI)	Índice vetorial para busca semântica (RAG)	Gratuito (local)
Maritaca AI (Sabiá-3)	LLM em português para análise profunda	~R\$0,01/mês
tlextract / validators	Análise de URLs suspeitas	Gratuito (local)
joblib	Serialização do modelo ML em disco	Gratuito (local)
FastAPI + uvicorn	API REST com documentação automática	Gratuito

2.4 Scraping

Tecnologia	Finalidade
Telethon	Cliente para a API do Telegram (MTProto)
APScheduler	Agendamento de tarefas (execução a cada hora)
httpx	Envio dos dados coletados para o Backend

2.5 Infraestrutura

Tecnologia	Finalidade
Docker	Containerização dos serviços
Docker Compose	Orquestração dos containers
PostgreSQL 15	Banco de dados relacional

Git / GitHub	Controle de versão
--------------	--------------------

3. Estrutura do Repositório

O projeto é organizado como um monorepo:

```
S07-dAurora-Anatel/
├── frontend/           # Aplicação Next.js
├── backend/            # API FastAPI
├── IA/                 # Módulo de inteligência artificial
├── scraping/           # Módulo de scraping do Telegram
├── Documentacao/       # Documentos do projeto
├── docker-compose.yaml
├── .env                # Variáveis de ambiente (não versionado)
└── README.md
```

3.1 Frontend

```
frontend/
├── app/
│   ├── dashboard/      # Tela principal
│   ├── list/           # Lista de casos
│   ├── case/[id]/      # Detalhes de um caso
│   ├── manual_analysis/ # Análise manual
│   ├── components/     # Componentes reutilizáveis
│   └── services/api.js  # Centralização das chamadas HTTP
├── public/             # Imagens e ícones estáticos
├── Dockerfile
└── package.json
```

3.2 Backend

```
backend/app/
├── api/
│   ├── casos.py        # CRUD + filtros de casos
│   ├── analyses.py     # CRUD de análises
│   ├── usuarios.py     # CRUD de usuários
│   └── analise_manual.py # Integração com IA
├── core/database.py    # Configuração SQLAlchemy
├── models/models.py    # Modelos das tabelas
├── schemas/schemas.py # Schemas Pydantic
├── services/ia_service.py # Cliente HTTP para a IA
└── main.py             # Entrypoint FastAPI
```

3.3 Módulo de IA

```
IA/discovery_ai/
├── api/main.py          # API REST (FastAPI) - v2
├── rag/
│   ├── rag_engine.py    # Motor RAG: embeddings + FAISS
│   └── sabia_client.py  # Integração Maritaca AI (Sabiá-3)
├── classifier/ml_classifier.py # TF-IDF + Regressão Logística
├── analyzer/
│   ├── link_analyzer.py # Análise heurística de URLs
│   ├── detection_service.py # Orquestrador do pipeline
│   └── situation_analyzer.py # Análise em linguagem natural
├── config/settings.py   # Carregador de configurações
├── data/
│   ├── config.json      # Parametrização central
│   ├── knowledge_base/taxonomia_golpes.md # Base RAG
│   ├── vector_store/    # Índice FAISS (auto-gerado)
│   └── classifier_model.joblib # Modelo ML treinado (auto-gerado)
├── scripts/
│   ├── setup_and_test.py # Setup inicial e smoke tests
│   └── evaluation_report.py # Relatório P/R/F1
```


4. Configuração do Ambiente

4.1 Pré-requisitos

Requisito	Versão Mínima
Docker Desktop	4.x
Git	2.x
Node.js (desenvolvimento local)	18.x
Python (desenvolvimento local)	3.10
Conta Telegram (para scraping)	—
Conta Maritaca AI (para Sabiá)	—

4.2 Variáveis de Ambiente

O projeto utiliza três arquivos .env:

.env (raiz)

```
DATABASE_URL=postgresql://usuario:senha@db:5432/nome_banco
POSTGRES_USER=usuario
POSTGRES_PASSWORD=senha
POSTGRES_DB=nome_banco
```

IA/.env

```
MARITACA_API_KEY=sua_chave_aqui
MARITACA_MODEL=sabia-3
API_HOST=0.0.0.0
API_PORT=8001
RAG_VECTOR_STORE_PATH=./data/vector_store
RAG_KNOWLEDGE_BASE_PATH=./data/knowledge_base
EMBEDDING_MODEL=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
CLASSIFIER_PATH=./data/classifier_model.joblib
```

scraping/.env

```
TELEGRAM_API_ID=seu_api_id
TELEGRAM_API_HASH=seu_api_hash
TELEGRAM_PHONE=+55seu_numero
BACKEND_URL=http://backend:8000
```

4.3 Executando o Projeto

1. Clone o repositório:

```
git clone <url>
```

2. Configure os arquivos .env conforme descrito acima.

3. Execute a autenticação do Telegram (primeira vez apenas):

```
python -c "from telethon.sync import TelegramClient; import os; from dotenv import
load_dotenv; load_dotenv(); client = TelegramClient('discovery_session',
int(os.getenv('TELEGRAM_API_ID')), os.getenv('TELEGRAM_API_HASH'));
client.start(phone=os.getenv('TELEGRAM_PHONE')); client.disconnect()"
```


4. Inicie todos os serviços:

```
docker-compose up --build
```

Serviços disponíveis após a inicialização:

Serviço	URL
Frontend	http://localhost:3000
Backend (API)	http://localhost:8000
Backend (Docs)	http://localhost:8000/docs
IA (API)	http://localhost:8001
IA (Docs)	http://localhost:8001/docs

5. Banco de Dados

5.1 Modelo de Dados

O PostgreSQL é composto por quatro tabelas principais:

- casos — registra cada conteúdo analisado (texto ou URL)
- analises — armazena os resultados da análise de IA para cada caso
- usuarios — usuários do sistema
- exportacoes — registra os PDFs exportados por cada usuário

5.2 Tabelas

casos

Coluna	Tipo	Descrição
id	INTEGER PK	Identificador único do caso
data_publicacao	TIMESTAMP	Data/hora de registro do caso
fonte	VARCHAR	Origem do conteúdo (manual, telegram, etc.)
titulo	VARCHAR	Tipo de golpe identificado
url	VARCHAR	URL analisada (quando aplicável)
legenda	TEXT	Texto completo do conteúdo analisado
situacao	VARCHAR	Nível de risco (ALTO, MÉDIO, INDEFINIDO)

analises

Coluna	Tipo	Descrição
id	INTEGER PK	Identificador único da análise
caso_id	INTEGER FK	Referência ao caso analisado
usuario_id	INTEGER FK	Referência ao usuário (opcional)
veracidade	NUMERIC	Probabilidade de ser golpe (0 a 1)
erros_ortograficos	INTEGER	Quantidade de erros ortográficos detectados
uso_ia_generativa	INTEGER	Indicadores de IA generativa detectados
links_suspeitos	INTEGER	Quantidade de links suspeitos
taxonomia_repetitiva	INTEGER	Padrões taxonômicos repetitivos
tipo	VARCHAR	Tipo de análise (manual, manual_url)

usuarios

Coluna	Tipo	Descrição
id	INTEGER PK	Identificador único do usuário
nome	VARCHAR	Nome do usuário
funcao	VARCHAR	Função/cargo do usuário

exportacoes

Coluna	Tipo	Descrição
id	INTEGER PK	Identificador único da exportação
caso_id	INTEGER FK	Referência ao caso exportado
usuario_id	INTEGER FK	Referência ao usuário que exportou
data_exportacao	TIMESTAMP	Data/hora da exportação

6. API Backend

Documentada automaticamente pelo FastAPI em <http://localhost:8000/docs>.

6.1 Casos

Método	Endpoint	Descrição
GET	/casos	Lista todos os casos com filtros opcionais
GET	/casos/recentes	Retorna os 3 casos mais recentes
GET	/casos/estatisticas	Estatísticas por situação (com filtros)
GET	/casos/tipos-golpe	Lista os tipos de golpe cadastrados
GET	/casos/{id}	Detalhes de um caso específico
POST	/casos	Cria um novo caso manualmente

6.2 Análises

Método	Endpoint	Descrição
GET	/analises	Lista todas as análises
GET	/analises/caso/{caso_id}	Análises de um caso específico
GET	/analises/{id}	Detalhes de uma análise
POST	/analises	Cria uma nova análise

6.3 Análise Manual

Método	Endpoint	Descrição
POST	/analise-manual/texto	Analisa um texto via IA e salva no banco
POST	/analise-manual/url	Analisa uma URL via IA e salva no banco

6.4 Parâmetros de Filtro — GET /casos

Parâmetro	Tipo	Descrição
titulo	string	Filtra por título (busca parcial, case-insensitive)
fonte	string	Filtra por fonte (manual, telegram, etc.)
tipo_golpe	string	Filtra por tipo de golpe
data_inicio	string (YYYY-MM-DD)	Filtra casos a partir desta data
data_fim	string (YYYY-MM-DD)	Filtra casos até esta data

7. Módulo de Inteligência Artificial

7.1 Pipeline de Análise

Cada entrada (texto, URL ou situação) passa por 5 etapas sequenciais:

Etap a	Módulo	Descrição
1	ML Classifier	TF-IDF + Logistic Regression. Local, ~5ms, 12 classes + legítimo.
2	Link Analyzer	Heurísticas: typosquatting, encurtadores, TLD suspeito, HTTPS, subdomínios. ~2ms.
3	RAG Engine	sentence-transformers → FAISS. Recupera trechos da base de conhecimento. ~50ms.
4	Sabiá-3 (Maritaca AI)	LLM em português. Análise profunda com contexto RAG. ~1-2s.
5	Fusão	Consolida resultados de todas as etapas em resposta unificada com risco final.

7.2 Classificação de Risco

O nível final é determinado pela fusão dos resultados do ML e do Sabiá, com precedência para o nível mais alto:

Nível	Critério
ALTO	Confiança $\geq$ 80% ou presença de múltiplos indicadores críticos
MÉDIO	Confiança entre 60% e 79%
BAIXO	Confiança entre 30% e 59%
SEGURO	Confiança $<$ 30% — conteúdo classificado como legítimo

7.3 Tipos de Golpe Suportados (12 classes)

Código	Label	Nome
GOLPE_01	phishing	Phishing por E-mail ou Mensagem
GOLPE_02	golpe_pix	Golpe do PIX / Falsa Central de Atendimento
GOLPE_03	falso_emprestimo	Falso Empréstimo / Antecipação FGTS
GOLPE_04	falsa_promocao	Falsa Promoção / Sorteio
GOLPE_05	falso_servico	Falso Serviço / Prestador
GOLPE_06	engenharia_social	Engenharia Social / Falso Funcionário
GOLPE_07	clonagem_conta	Clonagem de WhatsApp / Redes Sociais
GOLPE_08	falso_investimento	Falso Investimento / Pirâmide
GOLPE_09	golpe_suporte_tecnico	Romance Scam / Falso Relacionamento
GOLPE_10	golpe_qr_code	Link Malicioso / Malware

GOLPE_11	golpe_delivery	Golpe de Delivery / Compra
—	legítimo	Conteúdo Legítimo

7.4 Métricas de Avaliação (v2)

Avaliação	Resultado
Acurácia hold-out (20%)	72,22%
F1 cross-validation 5-fold	76,75% ± 11,05%
Acurácia nos dados de treino (ML Classifier)	88%
Acurácia conjunto manual	100% (19/19)
Falsos positivos (conjunto manual)	0
Falsos negativos (conjunto manual)	0

7.5 Endpoints da API de IA

Análise

Método	Rota	Descrição
POST	/api/analyze	Análise completa de texto ou mensagem
POST	/api/analyze/situation	Análise de situação em linguagem natural
POST	/api/analyze/url	Análise de URL específica
POST	/api/analyze/batch	Análise em lote (ML local, sem custo de API)

Administração

Método	Rota	Descrição
GET	/api/health	Health check
GET	/api/taxonomy	Lista todos os tipos de golpe
GET	/api/stats	Estatísticas da sessão
GET	/api/evaluation	Relatório de acurácia (precision/recall/F1)
GET	/api/config	Lê configurações atuais
POST	/api/config/reload	Recarrega config.json sem reiniciar
POST	/api/rag/rebuild	Reconstrói índice FAISS
POST	/api/classifier/retrain	Re-treina o classificador ML

7.6 Exemplo de Resposta

```
{
  "success": true,
  "data": {
    "is_suspicious": true,

```

```

    "risk_level": "ALTO",
    "fraud_type": "phishing",
    "fraud_code": "GOLPE_01",
    "confidence": 0.89,
    "explanation": "Mensagem típica de phishing com urgência artificial...",
    "recommendations": ["Nao clique no link", "Contate o banco pelo numero
oficial"],
    "indicators": ["Urgencia artificial", "Solicitacao de dados"],
    "analysis_time_ms": 12
  }
}

```

7.7 Parametrização (config.json)

Edite IA/data/config.json para ajustar comportamentos sem mexer no código:

```

{
  "risk_thresholds": {
    "alto": { "suspicious_prob_min": 0.80, "confidence_min": 0.60 },
    "medio": { "suspicious_prob_min": 0.60, "confidence_min": 0.30 }
  },
  "modules": {
    "use_sabia_by_default": false,
    "use_link_analyzer": true,
    "use_rag": true
  },
  "rag": { "top_k_chunks": 3, "chunk_size": 400 }
}

```

Após editar, recarregue sem reiniciar:

```
curl -X POST http://localhost:8001/api/config/reload
```

7.8 Expandindo a Base de Conhecimento

Para adicionar novos tipos de golpe ao RAG:

- Edite IA/data/knowledge_base/taxonomia_golpes.md
- Reconstrua o índice: POST /api/rag/rebuild

Para adicionar novos exemplos ao classificador ML:

- Edite TRAINING_DATA em IA/classifier/ml_classifier.py
- Re-treine: POST /api/classifier/retrain

8. Módulo de Scraping

8.1 Funcionamento

Coleta mensagens de canais e grupos públicos do Telegram via Telethon (protocolo MTProto).

Fluxo de execução:

- O serviço inicia junto com os demais via Docker Compose e executa o primeiro ciclo imediatamente.
- O APScheduler agenda a execução a cada hora.
- Para cada grupo monitorado, busca as 20 mensagens mais recentes.
- Cada mensagem é enviada ao endpoint POST /analise-manual/texto do Backend.
- O Backend aciona a IA, processa e salva o resultado no banco.

8.2 Autenticação

O Telethon requer autenticação via número de telefone na primeira execução. O arquivo de sessão gerado (discovery_session.session) é persistido no volume Docker e reutilizado nas execuções seguintes.

ATENÇÃO: O arquivo `discovery_session.session` contém credenciais de acesso ao Telegram e não deve ser versionado. Ele está incluído no `.gitignore`.

8.3 Configuração dos Grupos Monitorados

Os grupos monitorados são configurados em `scraping/scrapper.py`:

```
GRUPOS_MONITORADOS = [  
    "username_do_grupo_1",  
    "username_do_grupo_2",  
]
```

O username corresponde ao nome exibido após `t.me/` no link de convite público.

9. Segurança e Boas Práticas

9.1 Variáveis de Ambiente

- Todas as credenciais são armazenadas em arquivos `.env`, nunca no código-fonte.
- Os arquivos `.env` estão no `.gitignore` e não são versionados.
- Um arquivo `.env.example` é fornecido como template para novos desenvolvedores.

9.2 CORS

O Backend configura o CORS para aceitar requisições apenas do frontend em `localhost:3000`. Em produção, este valor deve ser atualizado para o domínio real da aplicação.

9.3 Healthchecks

O Docker Compose implementa healthchecks para garantir a ordem correta de inicialização:

Serviço	Aguarda
backend	IA estar healthy (GET /api/health retornar 200)
scraping	Backend estar healthy (GET / retornar 200)

10. Estratégia de Branches

Branch	Finalidade
main	Código estável em produção
homologacao	Ambiente de testes pré-produção
feature/*	Desenvolvimento de novas funcionalidades

Fluxo recomendado:

- Criar branch feature/* a partir de main
- Desenvolver e testar localmente
- Abrir Pull Request para homologacao
- Testar em homologacao
- Abrir Pull Request para main

11. Glossário Técnico

Termo	Definição
API REST	Interface de programação baseada em HTTP para comunicação entre serviços.
Docker / Docker Compose	Plataforma de containerização que empacota aplicações em ambientes isolados.
FAISS	Biblioteca da Meta para busca eficiente em espaços vetoriais de alta dimensão.
FastAPI	Framework Python para criação de APIs REST com tipagem estática e documentação automática.
Healthcheck	Verificação periódica do estado de saúde de um serviço Docker.
LLM	Large Language Model — modelo de linguagem de grande escala (ex: Sabiá-3).
MTPROTO	Protocolo de comunicação proprietário do Telegram usado pelo Telethon.
ORM	Object-Relational Mapping — mapeamento entre objetos Python e tabelas do banco.
RAG	Retrieval-Augmented Generation — enriquece o prompt do LLM com contexto vetorial.
SQLAlchemy	ORM Python para interação com bancos de dados relacionais.
Telethon	Biblioteca Python para usar a API oficial do Telegram.
TF-IDF	Term Frequency-Inverse Document Frequency — representação vetorial de textos para ML.
Uvicorn	Servidor ASGI assíncrono utilizado para executar aplicações FastAPI.
Volume Docker	Mecanismo de persistência de dados entre reinicializações de containers.